

PyBaMM

Training day 1: PyBaMM basics and practical simulations

PyBaMM Conference 2026 · Ann Arbor, Michigan · 15 October 2026

Today's session

30 min

Talk: PyBaMM basics and an overview of each notebook

60 min

Self-guided notebooks, with the PyBaMM team on hand to help

- 01 Getting started — models, parameters, experiments and outputs
- 02 Power vs energy cells — C-rate sweeps and electrode thickness
- 03 Long experiments — hundreds of cycles and summary variables
- 04 Reference performance tests — ageing cycles with periodic RPTs
- 05 Custom experiments — custom terminations and control

Set up before we start

Clone the repository › **Install the requirements** › **Open training/day_1**

```
git clone https://github.com/pybamm-team/pybamm-con-2026
cd pybamm-con-2026
pip install -r requirements.txt
jupyter lab
```

Stuck installing? Ask now: the notebooks need a working environment.

Getting started with PyBaMM

Notebook 01

What is PyBaMM?



A modelling framework

For writing and solving the differential equations that describe a battery.



A library

Battery models from the SPM to the DFN, with options for thermal effects and degradation, and published parameter sets for many cells.



Experiment tools

Lab-style protocols written in plain English, long cycling studies, summary variables and plotting.

Every simulation follows the same pipeline



- Model: the equations for the cell, with no numbers yet
- Parameter values: fill in the numbers for a particular cell
- Geometry, mesh and discretisation: turn the equations into a system that can be solved numerically
- Solver: steps the equations forward in time

Running a simulation

```
import pybamm

model = pybamm.lithium_ion.DFN()
sim = pybamm.Simulation(model)
sim.solve([0, 3600])
sim.plot()
```

- Doyle-Fuller-Newman (DFN) model of a lithium-ion cell
- One hour constant-current discharge
- Voltage, plus concentrations and potentials through the electrodes and separator

Lithium-ion models



SPM

Each electrode represented by a single particle, capturing solid-state diffusion.



SPMe

The single particle model plus electrolyte concentration and potential.



DFN

The full porous-electrode model: particles throughout each electrode, coupled through the electrolyte.



Options

Thermal, SEI growth, lithium plating, particle cracking, loss of active material, particle size distributions and more.

Parameters

```
parameter_values = pybamm.ParameterValues("Chen2020")
parameter_values.search("particle radius")

parameter_values["Positive particle radius [m]"] = 3e-6
sim = pybamm.Simulation(
    model, parameter_values=parameter_values
)
```

- Published parameter sets for many cells and chemistries
- Parameters can be constants, functions (e.g. OCP vs stoichiometry) or interpolated data
- Search by keyword to find the parameter you want to change

Experiments in plain English

```
experiment = pybamm.Experiment(
    [
        (
            "Discharge at C/2 until 3.0V",
            "Rest for 1 hour",
            "Charge at 1C until 4.1V",
            "Hold at 4.1V until 50 mA",
        )
    ]
    * 2
)
sim = pybamm.Simulation(model, experiment=experiment)
sol = sim.solve()
```

- Current, C-rate, power, voltage and resistance control
- Terminations on voltage, current or time
- Drive cycles from measured current or power data

Reading the outputs

```
sol["Voltage [V]"](t)        # voltage at times t [s]
sol.cycles[0].steps[1]       # one step of one cycle
sol.summary_variables        # per-cycle metrics
```

Any variable, any time

Voltage, current, concentrations, potentials and temperatures at the times you choose.

Cycles and steps

Each cycle and step of an experiment can be analysed on its own.

Summary variables

Capacity, lithium inventory and active material per cycle, for degradation studies.

01 · Getting started

Run a model › Compare models › Read the outputs › Change parameters › Run an experiment › Write an equation

Exercises

- Compare the SPM and DFN at 3C with the Chen2020 parameters. Where do they disagree?
- Add a lumped thermal model and plot the cell temperature for a 1C and a 2C discharge
- Add a second equation to the decay model and check it against the exact solution

Solving practical simulations with PyBaMM

Notebooks 02 to 05

02 · Power vs energy cells

Sweep C-rate 0.1C to 20C › Energy and mean power per run › Halve electrode thickness › Compare the two cells

- Thick electrodes store more energy but lose it quickly at high rate
- Thin electrodes hold their capacity to much higher C-rates
- Exercise: define a cell mass and compute specific energy and power

03 & 04 · Long experiments and RPTs

```
experiment = pybamm.Experiment(
    [cycle] * 500, termination="80% capacity"
)
sol = sim.solve(save_at_cycles=5)
pybamm.plot_summary_variables(sol)

# ageing cycles followed by an RPT
rpt_sol = rpt_sim.solve(starting_solution=ageing_sol)
```

- Stop on capacity fade rather than cycle count
- Keep every nth cycle, or chosen cycles
- Alternate ageing cycles with reference performance tests

05 · Custom experiments

Custom terminations

End a step on any model variable, e.g. when the anode potential reaches 0.02 V, to avoid lithium plating.

Explicit control

Set the current as a function of other variables, e.g. constant power, I = P / V.

Implicit control

Give an equation the current must satisfy, e.g. hold the anode potential constant.

Over to you

Open training/day_1 and start with 01-getting-started

Self-guided session

New to PyBaMM?

Work through 01-getting-started, then 02 power vs energy.

Used it before?

Jump to 03 to 05: long cycling, RPTs and custom experiments. Or bring your own problem.

Stuck?

Put your hand up. Docs at docs.pybamm.org, discussions on the PyBaMM GitHub.

Tomorrow: the new PyBaMM model zoo, writing your own model, input parameters and solvers